UNIVERSIDAD DE CARABOBO
Facultad Experimental de Ciencias y Tecnología
Departamento de Computación
Unidad Académica de Algoritmos y Programación
CAO402: Algoritmos II

Fecha: 06/03/2024
Profesor: Álvaro Espinoza

PARCIAL I
Estructuras Lineales
(Valor: 20%)

Nombre y Apellido:_____ **Cédula de Identidad:** _____

PARTE I: selección simple. **(1 punto C/U).** Cada respuesta incorrecta resta 1/3 pts. Una respuesta en blanco no resta puntaje.

1. Se tiene una clase B que hereda de A, una clase D que hereda de C, y a su vez, A es contenido de D y D es contenido de B:
    a. D puede contener objetos de tipo B que contienen objetos de tipo D.
    b. D puede contener objetos de tipo B que contienen objetos de tipo C.
    c. Esta relación no es posible en la Programación Orientada a Objetos.
    d. D friendzonea a B.
    e. D puede contener objetos de tipo A que contienen objetos de tipo D.
2. Sea *A* una clase con todos sus atributos protegidos:
    a. Una clase *B* hereda de *A* sus atributos pero NO puede acceder a ellos directamente.
    b. Es posible modificar sus atributos.
    c. Una clase *C* externa que es compuesta por *A* puede acceder a sus atributos directamente.
    d. Honestamente me gusta más el cálculo.
    e. Una clase *B* hereda de *A* sus atributos pero SI puede acceder a ellos directamente.
3. Si una clase *A* contiene un atributo *x* que es un apuntador a un objeto de la misma clase *A*:
    a. *x* obligatoriamente tendría que ser privado al ser de la misma clase que su contenedor.
    b. Si *x* es público entonces las clases hijas de *A* no podrían tener acceso a él porque el atributo es de clase *A*.
    c. Estoy mareado por tanta clase por aquí y tanta clase por allá.
    d. Si *B* es una clase hija de *A*, entonces el atributo *x* ahora será un objeto de tipo *B* a causa de la herencia.
    e. Es posible para *A* acceder a los atributos de *x*desreferenciado en cualquier momento.
4. Dada una clase *A* que contiene de atributos a objetos de las clases *B* y *C* y estos son creados directamente en el constructor, esto sería un ejemplo claro de:
    a. Composición.
    b. Agregación.
    c. Polimorfismo.
    d. Este parcial es una agresión..
    e. Herencia.
5. Si *A* es una clase contenedor agregada de *B*, entonces:
    a. Cuando se crea un objeto de clase *B* se crea automáticamente un objeto de clase *A* que es contenido de *B*.
    b. El ciclo de vida de *A* es el mismo que el de *B*.
    c. *B* tiene acceso a los atributos privados de *A*.
    d. La destrucción de *A* no implica la destrucción de *B*.
    e. Las consecuencias de esta asociación son las mismas que si *A* fuera clase heredada de *B*.
6. Seleccione la verdadera:
    a. Una clase hija con todos los métodos de sus ancestros sobrecargados puede acceder a los métodos de su clase abuela.
    b. Se puede acceder a un atributo privado de la clase madre en una herencia usando métodos públicos/protegidos setters o getters.
    c. Algoritmos 1 es más fácil que algoritmos 2.
    d. Las clases hijas pueden usar atributos privados de los padres.
    e. Existe la posibilidad de que una clase madre pueda acceder a los atributos propios de la clase hija.
7. Se compila un programa y este falla, el error tiene que ver con dos métodos que se llaman igual para una misma clase y tienen los mismos parámetros. Este fallo se debe a:
    a. Polimorfismo Dinámico.
    b. Polimorfismo Estatico.
    c. Encapsulamiento.
    d. El compilador tiene flojera de leer tu programa.
    e. Uno de los métodos debería ser estático.
8. Dada una clase *A* que hereda de *B* y una clase *C* que compone a *A*:
    a. Es posible demostrar con métodos avanzados de inducción completa que esta opción no es la correcta.
    b. B puede contener objetos de clase C..
    c. El ciclo de vida de *C* es el mismo que el de *B*.
    d. A tiene acceso a los atributos/métodos privados de C..

    e. Solo A puede contener objetos de tipo C.
9. Sea una clase A con una clase hija B y una clase C que "usa" a la clase A:
    a. Un objeto de clase B puede "usar" a un objeto de clase C.
    b. Un objeto de clase C solo puede "usar" objetos de clase A y no de clase B.
    c. Me la pase haciendo muchas cosas esta semana pero ninguna de ellas fue estudiar.
    d. Un objeto de clase C puede "usar" objetos de clase A y B.
    e. Un objeto de clase C puede "usar" objetos de clase B y no de clase A.
    f. Existe la posibilidad de que una clase madre pueda acceder a los atributos propios de la clase hija.
10. Un ejemplo de encapsulamiento es:
    a. El acceso a los atributos privados mediante métodos públicos.
    b. El miedo que le tengo al parcial de Grafos.
    c. El caso de que una clase realiza acciones mediante sus métodos cuyo funcionamiento es invisible al usuario.
    d. El hecho de tener múltiples métodos privados y públicos para manipular atributos.
    e. El caso de que una clase hija sólo pueda acceder a atributos públicos o protegidos de la clase madre.

PARTE II: Modelado de Clases: Realice el diagrama de clases correspondiente a la siguiente situación **(5 pts)**:

Pokimon Parte II

La empresa Gamefreakis le ha contratado para desarrollar su videojuego llamado Pokimon. Desarrollado por una compañía desconocida de algún país de Asia del Sur. A pesar de no ser oficial, ha ganado popularidad entre los aficionados a los juegos de rol y aventuras. La compañía pirata necesita desarrollar un sistema de gestión para organizar el juego y mantener un seguimiento de los jugadores y sus actividades. El sistema debe permitir el registro de diferentes aspectos del juego, como batallas entre jugadores, intercambio de Pokimon, exploración de mapas y participación en eventos especiales. Cada aspecto tiene información detallada, incluyendo la ubicación en el juego, fecha y hora, tipo de actividad y una descripción.

Además de las actividades, el sistema debe gestionar la información de los jugadores del juego. Se requiere registrar datos como el nombre del jugador, nivel, Pokimon capturados, objetos encontrados y logros desbloqueados. Algunos jugadores pueden tener roles especiales dentro del juego, como líderes de gimnasio, maestros Pokimon o campeones de la liga. Las actividades del juego pueden incluir batallas entre jugadores, competiciones de captura de Pokimon, torneos de gimnasio y búsqueda de objetos raros. Cada actividad tiene detalles específicos, como el nombre, nivel de dificultad, recompensas y requisitos de participación.

El sistema también debe permitir la asignación de recursos y responsabilidades para cada actividad, incluyendo la designación de árbitros, ayudantes y personal de soporte técnico. Cada miembro del equipo tiene una función específica en la actividad y puede estar asignado a múltiples actividades. Además, la compañía pirata necesita mantener un registro de los Pokimon disponibles en el juego, incluyendo detalles como el tipo, habilidades, nivel de evolución y estadísticas de combate. Se desea realizar un seguimiento del número de capturas de cada Pokimon y las estrategias más utilizadas en las batallas.

PARTE III: Implementación (5 pts):

Implemente las clases necesarias para modelar un sistema de relojes. Se requiere la clase Reloj, cuyos datos a tener en cuenta son el tipo de reloj (analógico o digital), marca del reloj, hora actual, minuto actual y segundo actual. Se requiere que el reloj tenga la capacidad de "avanzar en el tiempo". Debe implementar los siguientes métodos: *`avanzarTiempo(`**`entero:`** `horas, minutos, segundos)`*: avanza el tiempo del reloj en la cantidad de horas, minutos y segundos especificados. Ejemplo: 10:00:20 si avanzamos 10:00:30 la hora nueva sería 20:00:40. Este método debe ser a lo sumo O(1).

Por otra parte, también se requiere la clase ZonaHoraria. Dicha clase contendrá los siguientes atributos: nombre de la zona horaria, lista de los relojes que pertenecen a ella (usar arreglos) y el número de relojes presentes. Se pide que implemente los métodos: `agregarReloj(Reloj: reloj)`: agrega un reloj nuevo al arreglo de relojes. `eliminarRelojPorNumero(`**`entero:`** `n)`: elimina un reloj del arreglo de relojes. `sincronizarRelojes(`**`entero:`** `horas, minutos, segundos)`: sincroniza a todos los relojes en la hora especificada.

"La Ciencia de la Computación no tiene que ver con las computadoras más que la Astronomía con los telescopios." **- Edsger Dijsktra.**